



Algorithms better than LPT for semi-online scheduling with decreasing processing times

T.C.E. Cheng^a, Hans Kellerer^{b,*}, Vladimir Kotov^c

^a Department of Logistics and Maritime Studies, The Hong Kong Polytechnic University, Kowloon, Hong Kong

^b Institut für Statistik und Operations Research, Universität Graz, Universitätsstraße 15, A-8010 Graz, Austria

^c Belarussian State University, Faculty of Applied Mathematics and Computer Science, Prospekt Nezavisimosti 4, 220030 Minsk, Belarus

ARTICLE INFO

Article history:

Received 20 January 2012

Received in revised form

8 May 2012

Accepted 16 May 2012

Available online 4 June 2012

Keywords:

Online algorithms

Semi-online algorithms

Competitive ratio

Multiprocessor scheduling

ABSTRACT

We consider the semi-online multiprocessor scheduling problem with m identical, parallel machines to minimize the makespan, where the jobs arrive in decreasing order of processing times. The famous Longest Processing Time (LPT) algorithm by Graham (1966) [4] for the classical offline multiprocessor scheduling problem schedules the jobs in decreasing order of processing times and has a worst-case bound of $4/3 - 1/(3m)$. So far, no algorithm with a better competitive ratio than the LPT algorithm has been given for the semi-online scheduling problem with decreasing processing times. In this note, we present a $5/4$ -competitive algorithm for $m \geq 3$ and an algorithm that is the best possible for $m = 3$, i.e. an algorithm with competitive ratio $(1 + \sqrt{37})/6$.

© 2012 Elsevier B.V. All rights reserved.

1. Introduction

In the classical multiprocessor scheduling problem, there are given n independent jobs with nonnegative processing times $p_1, \dots, p_n$ that have to be scheduled nonpreemptively on m identical, parallel machines $M_1, \dots, M_m$ with the objective of minimizing the makespan. In the *online* version of the multiprocessor scheduling problem, each job must be immediately and irrevocably assigned to one of the machines without any knowledge of future jobs. The performance of an online algorithm is measured by its *competitive ratio*, i.e., the worst-case ratio with respect to the optimum solution of the corresponding offline problem. In the sequel we will denote by C^* the value of an optimal solution of the offline problem. The total sum of the job processing times assigned to a machine M is called the *load* of machine M , and is denoted by $\ell(M)$. Moreover, we will identify jobs with their corresponding processing times, i.e., a job greater than x means a job with processing time greater than x .

The online multiprocessor scheduling problem was first investigated by Graham, who showed that the list scheduling algorithm has a competitive ratio of exactly $2 - 1/m$ [5,4] and it is the best possible for $m \leq 3$ [2]. A long list of improved algorithms

has since been published. The best heuristic is due to Fleischer and Wahl [3]. They designed an algorithm with a competitive ratio smaller than 1.9201, when the number of machines tends to infinity.

In practice, problems are often not really online or offline but somewhere in between. This means that, with respect to the online problem, some further information about the jobs is available. This additional information allows the improvement of the performance of the best solution algorithm with respect to the online problem. These problems are often vaguely called *semi-online problems*. We will investigate the *semi-online scheduling problem with decreasing processing times* (SOSDP), i.e., we know that $p_j \geq p_{j+1}$ for all $j \geq 1$.

It is well-known that the offline multiprocessor scheduling problem is strongly NP-hard by reduction from the 3-partition. The most popular heuristic is the *Longest Processing Time* algorithm (LPT for short) proposed by Graham [4]. The LPT algorithm sorts jobs in decreasing order of processing times and assigns them one at a time in this order to the machine with the smallest current load. The LPT algorithm has a worst-case performance ratio of $4/3 - 1/(3m)$. Obviously, LPT can be used as an algorithm for SOSDP. Many other approximation algorithms for offline multiprocessor scheduling have been developed and analyzed. Let us especially mention the paper by Hochbaum and Shmoys [6], which provides a PTAS for this problem by using a family of dual approximation algorithms.

The problem SOSDP was first studied by Seiden et al. [7]. They show that LPT with a competitive ratio of $7/6$ for $m = 2$ is

* Corresponding author.

E-mail addresses: lgtcheng@polyu.edu.hk (T.C.E. Cheng), hans.kellerer@uni-graz.at (H. Kellerer), kotovVM@yandex.ru (V. Kotov).

best possible for two machines, and give a lower bound of $(1 + \sqrt{37})/6 \approx 1.18046$ for $m \geq 3$. So far, no algorithm with a better competitive ratio than LPT has been found for SOSDP.

Problem SOSDP on two uniformly related machines is investigated by Epstein and Favrholdt [1]. They show that LPT has optimal competitive ratio $\frac{1}{4}(1 + \sqrt{17})$ overall, but does not have optimal competitive ratio for every value of the speed ratio between the two machines. They also construct different algorithms of optimal competitive ratio for the intervals where LPT fails.

In this note, we present a $5/4$ -competitive algorithm for $m \geq 3$ and a best possible algorithm for $m = 3$, i.e. an algorithm with a competitive ratio $(1 + \sqrt{37})/6$. Both algorithms are elementary and have linear running time. Essentially, they assign, if possible, a job to a machine for which the load will not exceed some lower bound multiplied by the competitive ratio; otherwise they assign the job to the machine with the smallest load.

2. A $5/4$ -competitive algorithm for an arbitrary number of machines

In this section we will present an algorithm, denoted by A , for SOSDP with a competitive ratio $5/4$. Since the LPT algorithm has a performance ratio $4/3 - 1/(3m)$, it outperforms algorithm A only for $m \leq 3$. The algorithm can be described as follows:

Algorithm A. 1. Assign job p_j to machine $M_j, j = 1, \dots, m$.
2. Set $L := p_m + p_{m+1}$.
3. Assign the current job $p_j, j \geq m + 1$, to the machine M with the largest load $\ell(M)$ such that $\ell(M) + p_j \leq \frac{5}{4}L$. Otherwise, assign it to the machine with the smallest load.

Theorem 1. *Algorithm A is a deterministic algorithm for SOSDP with competitive ratio $5/4$.*

Proof. W.l.o.g. assume that $C^* = 1$. The proof is established by contradiction. Then there is a counterexample with a *failure job* z that shall be the first job being assigned by A to a machine M such that $\ell(M) > 5/4$. Moreover, we assume that the counterexample is *minimal*, i.e., there is no counterexample with a smaller number of jobs. Consequently, z is the last job, i.e. $z = p_n$.

The jobs are divided into several classes according to their processing times: Let $X_1 := \{p_j \mid p_j > 3/4\}$, $X_2 := \{p_j \mid 3/4 \geq p_j > 1/2\}$, $X_3 := \{p_j \mid 1/2 \geq p_j > 1/4\}$ and $X_4 := \{p_j \mid p_j \leq 1/4\}$. Denote by x_i the cardinality of set $X_i, i = 1, \dots, 4$. Since the failure job z is assigned to the machine M with the smallest load, z has processing time greater than $1/4$ and $x_4 = 0$. Moreover, at most 3 jobs are assigned to each machine in the optimal solution.

Let $\mathcal{M}_1^*, \mathcal{M}_2^*$ and $\mathcal{M}_3^*$ denote the machines in the optimal solution with one, two and three jobs, respectively. The numbers m_1^*, m_2^*, m_3^* shall denote the corresponding cardinalities.

In the case that at most two jobs are assigned to each machine in the optimal solution, we have $C^* \geq \max_{j=1, \dots, m} \{p_j + p_{2m-j+1}\}$. It is obvious that in this case Algorithm A is $5/4$ -competitive. Thus, $m_3^* > 0$ and $z \leq 1/3$ hold.

Let n_v denote the number of jobs greater than v . Then,

$$n_{1/2} \leq m_1^* + m_2^*. \quad (1)$$

Because each job is greater than $1/4$, the second job on each of the machines in $\mathcal{M}_3^*$ is smaller than $3/8$. Then,

$$n_{3/8} \leq m_1^* + 2m_2^* + m_3^* = m + m_2^*. \quad (2)$$

We will distinguish two cases

Case 1. $p_1 + p_{m+1} \leq 5/4L$:

Set $\mathcal{M}_1 := \{M_1, \dots, M_{m_1^*+m_2^*}\}$ and $\mathcal{M}_2 := \{M_{m_1^*+m_2^*+1}, \dots, M_m\}$. Due to $p_1 + p_{m+1} \leq 5/4L$ Algorithm A assigns the m_2^* jobs $p_{m+1}, \dots, p_{m+m_2^*}$ to machines in $\mathcal{M}_1$.

By (1) all jobs of $\mathcal{M}_2$ have processing time less than or equal to $1/2$ and by (2) all second jobs of $\mathcal{M}_2$ have processing time less than or equal to $3/8$. Since at least $2(m_1^* + m_2^*)$ jobs (except z) are assigned to the machines of $\mathcal{M}_1$, there is at least one machine of $\mathcal{M}_2$, which has (at most) two jobs immediately before z is processed. But then z can be assigned as a third job to that machine, since the load would not exceed $1/2 + 3/8 + 1/3 = 29/24 < 5/4$, contradicting that z is a failure job.

Case 2. $p_1 + p_{m+1} > 5/4L$

If there is a machine in the optimal solution, which contains only one job, denoted by x , we can exchange x with p_1 , so that p_1 is alone on a machine without that the makespan of the optimal solution is increasing. Create a new instance by removing one machine and all jobs except z , which are together with p_1 on M_1 in the heuristic solution. The makespan of the optimal solution of the new instance will not increase. Because $p_1 + p_{m+1} > 5/4L$, job p_{m+1} is not assigned to M_1 . Thus, L will not change in the new instance and all jobs except z are assigned to the same machines as in the original instance. Only z is assigned to a different machine (if it was processed on M_1 before). In any case, the makespan will remain the same or it will possibly increase in the case that z was assigned to M_1 . Since our counterexample is minimal, we get $m_1^* = 0$. From $z > 1/4$, we conclude $x_1 = 0$.

We distinguish two subcases:

(2a) $5/4L \geq 1$

Let M_1^* denote the machine where p_1 is processed in the optimal solution. From $p_1 + p_{m+1} > 5/4L \geq 1$ follows that $p_1 > 1/2$, and we have $M_1^* \in \mathcal{M}_2^*$. Thus, there is a largest job p_ℓ with $p_1 + p_\ell \leq 1$. W.l.o.g. $p_\ell \in M_1^*$ and M_1^* consists of the two jobs p_1 and p_ℓ .

From $p_1 + p_{m+1} > 1$ we get $\ell > m + 1$ and with $5/4L \geq 1$ follows that there is a job $p_{\ell'}$, $\ell \geq \ell' > m + 1$, which is assigned by Algorithm A to machine M_1 such that $p_1 + p_{\ell'} \leq 5/4L$. Get a new instance by removing jobs of M_1 , i.e., $p_1, p_{\ell'}$ plus possibly a third job, and one machine. Similarly as above, L will not change and Algorithm A will not improve. Consider the optimal solution for m machines. Remove machine M_1^* and replace $p_{\ell'}$ by p_ℓ in the original optimal assignment. Since $p_{\ell'} \geq p_\ell$, the makespan of this solution for the new instance is not larger than for the makespan of the optimal solution for the original instance. Again this yields a contradiction to the minimality of our counterexample.

(2b) $5/4L < 1$

Denote by $\mathcal{A}$ the assignment of jobs to machines in the heuristic solution immediately before the failure job z is assigned. Let m_1, m_2, m_3 denote the corresponding number of machines with one, two or at least three jobs in $\mathcal{A}$.

If there is a machine M in $\mathcal{A}$ with only one job, we have $\ell(M) + z \leq 3/4 + 1/3 < 5/4$. Thus, every machine contains at least 2 jobs before z arrives, i.e. $m_1 = 0$.

Assume there is a machine in $\mathcal{A}$ with exactly 2 jobs y_1, y_2 such that $y_1 \geq y_2$ and $y_1 \in X_3$. We get with $y_2 \leq p_{m+1} \leq L/2$ that $y_1 + y_2 + z \leq 1/2 + L/2 + 1/3 \leq 5/4$. Hence, $y_1 \in X_2$ and $m_2 \leq x_2$, which implies with $m = m_2 + m_3$ that

$$n = 2m_2 + 3m_3 + 1 = 3m - m_2 + 1 \geq 3m - x_2 + 1.$$

On the other side, every job of X_2 is assigned in the optimal solution to a unique machine, together with at most one other job. Hence, $n \leq 2x_2 + 3(m - x_2)$, which is equivalent to $n \leq 3m - x_2$. We get a contradiction to the above inequality. $\square$

3. A best possible algorithm for three machines

In this section we will present an algorithm for SOSDP for three machines with a competitive ratio $c := (1 + \sqrt{37})/6$. According to the lower bound given in [7], this algorithm is best possible

in terms of the competitive ratio. Formally, **Algorithm A3** can be described as follows:

Algorithm A3. 1. Assign jobs p_1, p_2, p_3 to machines M_1, M_2, M_3 .
2. Set $L_0 := \max\{p_1, p_3 + p_4\}$. If $p_1 + p_4 \leq cL_0$, assign p_4 to M_1 and p_5 to M_2 and the remaining jobs to the machine with smallest load.
3. If $p_1 + p_4 > cL_0$, assign p_4 to M_3 . Set $L := \max\{L_0, \min\{p_2 + p_5, p_3 + p_4 + p_5\}\}$. If M_1 contains only job p_1 and if $p_1 + p_j \leq cL$, assign the current job $p_j, j \geq 5$, to machine M_1 . Otherwise, assign p_j to the machine with smallest load.

Theorem 2. *There exists a deterministic algorithm for SOSDP for $m = 3$ with competitive ratio $c = (1 + \sqrt{37})/6 \approx 1.18046$.*

Proof. To distinguish between assignments of **Algorithm A3** and optimal assignments we denote the machines of the optimal assignment by M_1^*, M_2^*, M_3^* . We may identify a machine with the jobs assigned to that machine. W.l.o.g. p_1 is assigned to the first machine in an optimal assignment, i.e. $p_1 \in M_1^*$. W.l.o.g. $C^* = 1$ holds. Therefore, $\ell(M_i^*) \leq 1$ for $i = 1, 2, 3$ and the total processing time of the jobs is at most 3.

As in **Theorem 1**, the proof is established by contradiction. Then there is a counterexample with a *failure job* z that shall be the first job being assigned by **A3** to a machine M such that $\ell(M) > c$. W.l.o.g. z is the last job. Note that z is always assigned to a machine with the smallest load.

It can be easily seen that the bounds L_0, L calculated by **A3** are lower bounds on the optimal makespan, i.e. $L_0 \leq L \leq 1$, and that that **A3** yields the claimed bound for an instance with at most 5 jobs. Note that the competitive ratio c is the root of a quadratic equation. More precisely, we have

Observation O1. $(c - 1/3)c = 1$.

Because the total load of the machines is at most 3, there is a machine with load less than or equal to $1 - z/3$ before z is assigned. Since z is assigned to the machine with smallest load, we have $(1 - z/3) + z > c$, which results in

Observation O2. $z > \frac{3}{2}(c - 1) \approx 0.27069$.

Consequently, we get

Observation O3. All jobs have a processing time greater than $\frac{3}{2}(c - 1)$, each machine in the optimal solution has at most 3 jobs and the counterexample contains $n \leq 9$ jobs.

Assume that $n = 9$. After assigning 8 jobs, there is a machine with at most two jobs and load not exceeding $p_1 + p_4$. Thus, the makespan obtained by **A3** is at most $p_1 + p_4 + p_9$. Each machine of the optimal solution contains exactly 3 jobs. Thus, two jobs of $\{p_1, p_2, p_3, p_4\}$ are on one machine and we get $p_3 + p_4 + p_9 \leq 1$ and hence $p_4 + p_9 \leq 2/3$. Obviously, $p_1 + p_8 + p_9 \leq 1$, which gives with **(O2)** that $p_1 \leq 1 - 2z \approx 0.45862$. Therefore, $p_1 + p_4 + p_9 \leq 1 - 2z + 2/3 < c$, which contradicts the assumption of a counterexample. Recall that **A3** yields the claimed bound for at most 5 jobs. Thus, we have shown

Observation O4. The counterexample contains 6, 7 or 8 jobs.

Elementary calculations using **(O3)** and the fact that the load of each machine in the optimal solution is at most one, deliver the following bounds summarized in

Observation O5. If $n \geq 7$, then $p_3 \leq 1/2, p_7 \leq 1/3$ and $p_6 + p_7 \leq 2/3$.

If $n = 8$, then $p_2 \leq 1/2$ and $p_5 + p_8 \leq 2/3$.

Assume that $p_1 + p_4 \leq cL_0$. In this case $p_4 \in M_1$. Moreover, p_5 can be assigned to M_2 and p_6 to M_3 , respectively. We get from **(O5)** that $p_3 + p_6 + p_7 \leq 7/6 < c$ and p_7 fits on machine M_3 . If $n = 8$,

$p_2 + p_5 + p_8 \leq c$ and p_8 fits on machine M_2 . Since by **(O4)** a counterexample contains at most 8 jobs, there is no failure job and we have proven

Observation O6. $p_1 + p_4 > cL_0$, i.e., job p_4 is assigned to machine M_3 .

Let $A = \{a_1, a_2, \dots, a_r\}, B = \{b_1, b_2, \dots, b_s\}$ be two sets of nonnegative integers both sorted in decreasing order with $r \geq s$. We say that A dominates B if $a_1 \geq b_1, a_2 \geq b_2, \dots, a_s \geq b_s$.

Assume that M_1^* consists only of job p_1 . Then, all other jobs are processed on machines M_2^*, M_3^* in the optimal solution. Since $p_1 \in M_1$, the jobs processed on machines M_2, M_3 , denoted by $\mathcal{J}$, are dominated by the jobs on machines M_2^*, M_3^* , denoted by $\mathcal{J}^*$. Thus, the optimal solution for an instance with two machines and jobs of $\mathcal{J}$ is not larger than for an instance with two machines and jobs $\mathcal{J}^*$, which means it is at most one. For two machines LPT has a worst-case performance of $7/6$. Consequently, no subset of $\mathcal{J}$ has a load greater than $7/6$ when assigned to two machines by LPT. But by **(O6)** no job will be assigned in Step 2 of **Algorithm A3** and the assignment of jobs to M_2 and M_3 by **A3** is identical to the LPT algorithm. Thus, immediately before z is assigned, there is a machine $M \in \{M_2, M_3\}$ with $\ell(M) + z \leq 7/6$. Since z is assigned to the machine with the smallest load, we get a contradiction to the assumption that z is the failure job. Applying **(O2)** we obtain

Observation O7. M_1^* contains at least two jobs and $p_1 \leq 1 - z \leq 0.73$.

Now assume that $L_0 \geq c - 1/3$. It follows with **(O1)** and **(O6)** that $p_1 + p_4 > cL_0 \geq 1$. Hence, $p_1 > 1/2$ and machine M_1^* contains exactly 2 jobs. Since $cL \geq cL_0 > 1$, the second job on M_1 is at least as large as the second job on M_1^* . Consequently, the jobs processed on machines M_2, M_3 are dominated by the jobs on machines M_2^*, M_3^* . Equivalently to the proof of **(O7)** we get a contradiction to the existence of a counterexample.

Observation O8. $L_0 < c - 1/3$.

Assume that $p_1 + p_6 > c$. Thus, M_1^* contains no job of the set $\{p_2, p_3, \dots, p_6\}$ and there is a machine in the optimal solution, which contains at least three jobs of $\{p_2, p_3, \dots, p_6\}$. This implies $p_6 \leq 1/3$. Since by **(O7)** $p_1 \leq 0.73$, we conclude that

Observation O9. $p_1 + p_6 \leq c$.

For the main part of the proof we distinguish two cases:

Case 1. $p_2 \geq p_3 + p_4$

Since $p_1 \geq p_2 \geq p_3 + p_4 > 0.54$ by **(O2)**, machines M_1^* and M_2^* contain at most 2 jobs and we have by **(O4)** that $n \leq 7$.

By the definition of L , $p_3 + p_4 + p_5 \leq L \leq 1$. Thus, if one the jobs p_5, p_6, p_7 is assigned by **A3** to machine M_3 , the load of M_3 is no larger than c . From **(O9)** we know that $p_2 + p_6 \leq c$ and if p_5 or p_6 is assigned by **A3** to machine M_2 the load of M_2 is also no larger than c . Finally, by **(O7)** machine M_1^* contains at least two jobs, which means $p_1 + p_7 \leq 1$. Thus, p_7 can be assigned to machine M_1 by **A3** without exceeding c . Since **A3** always assigns jobs to the machine with smallest load or to a machine for which the total load does not exceed c , none of the jobs $p_1, \dots, p_7$ can be a failure job.

Case 2. $p_2 < p_3 + p_4$

Since by **(O6)** $p_1 + p_4 > cL_0$, job p_4 is assigned to machine M_3 and we get that $L = \max\{p_1, p_2 + p_5, p_3 + p_4\} \leq 1$.

We distinguish several subcases depending on the locations of p_5 and p_6 .

Assume p_5 is assigned to M_1 . Thus, because of $p_2 < p_3 + p_4$, job p_6 must be processed on M_2 . By **(O9)** p_6 is not the failure job. By **(O5)** $p_7 \leq 1/3$ and p_7 can be assigned to machine M_3 since $p_3 + p_4 + p_7 \leq L_0 + 1/3 < c$ by **(O8)**. Job p_8 can be assigned to machine M_2 since due to **(O5)** $p_2 + p_5 + p_8 \leq 1/2 + 2/3 = 7/6 < c$.

Since $p_2 < p_3 + p_4$, it remains to consider that p_5 is assigned to M_2 . If p_6 is assigned to M_1 we proceed as above with p_7 and p_8 .

If p_6 is assigned to M_3 , then M_3 is the machine with smallest load after five jobs and we get from (O9) that $p_3 + p_4 + p_6 \leq p_1 + p_6 \leq c$. This implies $p_1 + p_7 \leq c$ and p_7 can be assigned to M_1 . Finally, p_8 can be assigned to M_2 since $p_2 + p_5 + p_8 \leq 1/2 + 2/3 = 7/6$ by (O5).

If p_6 is assigned to M_2 , as machine with smallest load, we get analogously $p_2 + p_5 + p_6 \leq p_1 + p_6 \leq c$ and $p_1 + p_7 \leq c$. Recall that $p_3 + p_4 \leq L_0$ and $p_8 \leq 1/3$ by (O5). Thus, job p_8 can be assigned to M_3 since due to (O8) $p_3 + p_4 + p_8 \leq L_0 + 1/3 \leq c$. This completes the proof of our theorem. $\square$

4. Conclusions

In this note we have presented a simple algorithm for problem SOSDP with a competitive ratio better than LPT and a best possible algorithm for SOSDP with three machines. For general m there is still a significant gap between $5/4$ and the lower bound $(1 + \sqrt{37})/6$. It is an interesting open question whether there are better lower bounds than $(1 + \sqrt{37})/6$, or whether for more than three machines, algorithms with competitive ratio $(1 + \sqrt{37})/6$ exist.

Acknowledgments

This research was partially supported by The Hong Kong Polytechnic University under grant number G-T397. The authors are thankful to an anonymous referee for his comments, which greatly improved the presentation of the paper.

References

- [1] L. Epstein, L.M. Favrholdt, Optimal non-preemptive semi-online scheduling on two related machines, *Journal of Algorithms* 57 (2005) 49–73.
- [2] U. Faigle, W. Kern, G. Turan, On the performance of on-line algorithms for partition problems, *Acta Cybernetica* 9 (1989) 107–119.
- [3] R. Fleischer, M. Wahl, On-line scheduling revisited, *Journal of Scheduling* 3 (2000) 343–353.
- [4] R.L. Graham, Bounds on multiprocessing timing anomalies, *SIAM Journal on Applied Mathematics* 17 (1969) 263–269.
- [5] R.L. Graham, Bounds for certain multiprocessor anomalies, *Bell System Technical Journal* 45 (1966) 1563–1581.
- [6] D.S. Hochbaum, D. Shmoys, Using dual approximation algorithms for scheduling problems: theoretical and practical results, *Journal of the Association for Computing Machinery* 34 (1987) 144–162.
- [7] S. Seiden, J. Sgall, G. Woeginger, Semi-online scheduling with decreasing job sizes, *Operations Research Letters* 27 (2000) 215–222.